

Rethinking motion keyframe extraction: a greedy procedural approach using a neural control rig

Théo Cheynel
Kinetix, LIX, École Polytechnique
France
theo@kinetix.tech

Omar El Khalifi
Kinetix
France
omar@kinetix.tech

Baptiste Bellot-Gurlet
Kinetix
France
baptiste.b@kinetix.tech

Damien Rohmer
LIX, École Polytechnique, IP Paris
France
drohmer@lix.polytechnique.fr

Marie-Paule Cani
LIX, École Polytechnique, IP Paris
France
marie-paule.cani@inria.fr

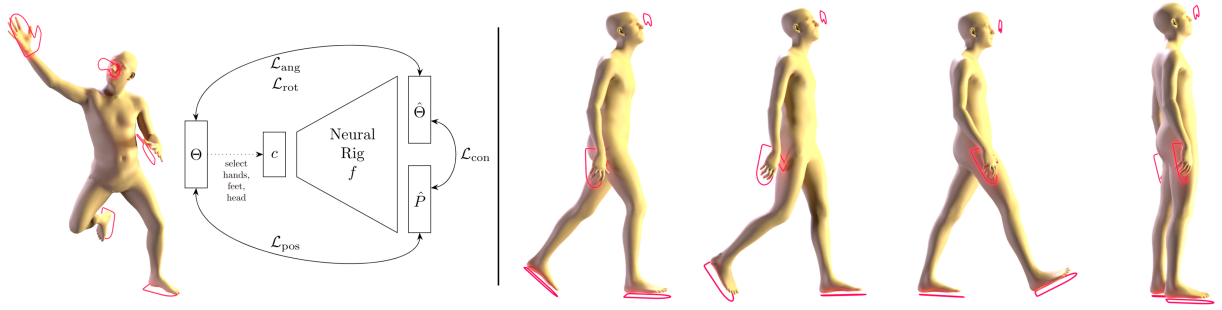


Figure 1: Overview of our neural rig architecture and results, and example of key poses from a reconstructed walking motion.

Abstract

3D animators traditionally use a “pose to pose” approach, whereas motion capture (MoCap) tools generate a pose for every frame, making the motion challenging to edit. We argue that current keyframe extraction methods are inadequate for human editing. To address this, we propose a novel approach that bridges the gap between MoCap animations and traditional 3D artist tools. Our contributions include learning a neural control rig as a differentiable proxy for more accurate key pose interpolation and formulating the task as an optimization problem, solved efficiently with a greedy dynamic programming algorithm.

ACM Reference Format:

Théo Cheynel, Omar El Khalifi, Baptiste Bellot-Gurlet, Damien Rohmer, and Marie-Paule Cani. 2024. Rethinking motion keyframe extraction: a greedy procedural approach using a neural control rig. In *SIGGRAPH Asia 2024 Posters (SA Posters '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3681756.3697960>

1 Introduction

We noticed a significant gap between statistical model outputs and the workflows of 3D animators, primarily due to the lack of datasets

of artist-made animation timelines. Motion understanding models typically represent character motion as sequences of successive poses, while 3D animators use the “pose-to-pose” principle [Johnston and Thomas 1995], blocking out main frames and interpolating between them. Moreover, animators rely on “control rigs,” which offer intuitive high-level control and more natural pose interpolation, instead of directly keyframing the “deform bones” which drive the skinning. This methodological difference makes the process of manually editing MoCap animations quite tedious, for tasks such as removing noise, modifying style, or adapting to specific scenes or characters.

Various methods exist for motion keyframe extraction, typically framed as selecting an optimal set of M points from an animation curve for the best possible interpolation. Iterative algorithms like Ramer-Douglas-Peucker and its variants perform well but are not optimal [Ramer 1972]. Roberts et al. [2018] use Dijkstra’s algorithm for an optimal set of frames, though its $O(Mn^2)$ complexity is slow. Heuristics, such as curve saliency [Bulut and Capin 2007], and statistical models, including PCA [Zhang et al. 2014], clustering [Zhang et al. 2013], and matrix factorization [Huang et al. 2005], are also shown to help identify keyframes.

Recently, machine learning has emerged as a solution for keyframe extraction. Genetic algorithms represent timelines as bit strings for flexible solutions [Liu et al. 2013], and Particle Swarm Optimization has been explored despite long running times [Chang et al. 2016]. A more efficient approach represents each frame as a graph, using deep RL on a graph neural network [Mo et al. 2021].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1138-1/24/12

<https://doi.org/10.1145/3681756.3697960>

We argue that the standard problem formulation limits output quality for several reasons. First, most methods minimize the error between interpolated and ground truth curves without considering the skeleton’s hierarchical structure. A better criterion would account for motion perception, such as errors in world-space joint position, rotation, or speed. Second, these methods only select pre-existing poses as keyframes, whereas using more extreme, non-original poses could improve reconstruction. Third, the results are not directly usable by artists who work with control rigs. Lastly, some methods process animation curves separately, which is less useful for human editing.

2 Method

Our method consists of two key elements : a “neural control rig” which approximates the behaviour of a standard rig, and a procedure to compute key frames by exploiting the differentiability of the rig and optimizing greedily the reconstruction.

2.1 Neural control rig

Conceptually, a control rig is a function $f : C \mapsto SO(3)^N$ which maps a controller vector $c \in C$ (usually, the position and rotation of a set of “controllers”) to the rotation of all N “deform bones”.

To mimic artists’ use of control rigs, we train a neural network to predict the rotation of the deform bones based on the position and rotation of a subset of bones (hands, feet, and head). We use the SMPL body model [Loper et al. 2015], as its shape parameter β allows for various skeletal morphologies.

The neural control rig takes as input a control vector c and the shape vector β , and is tasked with predicting the rotation of all bones $\hat{\Theta} = f_\beta(c) \in SO(3)^N$, as well as the position of all joints $\hat{P} = g_\beta(c) \in \mathbb{R}^{3 \times N}$, both expressed in the pelvis reference frame. Technically, there exists a function $J_\beta : \Theta \mapsto P$ that yields the world-space position of the joints based on the rotations; as such, the position information is redundant, but it helps make optimization straightforward in the algorithm detailed in section 2.2.

We optimize a weighted sum of several losses :

$$\begin{aligned} \mathcal{L}_{\text{con}} &= \frac{1}{N} \sum_{i=1}^N \|\hat{P}_i - J_\beta(\hat{\Theta}_i)\|_2 & \mathcal{L}_{\text{pos}} &= \frac{1}{N} \sum_{i=1}^N \|\hat{P}_i - J_\beta(\Theta_i)\|_2 \\ \mathcal{L}_{\text{rot}} &= \frac{1}{N} \sum_{i=1}^N \|J_\beta(\hat{\Theta}_i) - J_\beta(\Theta_i)\|_2 & \mathcal{L}_{\text{ang}} &= \frac{1}{N} \sum_{i=1}^N \alpha(\Theta_i^{-1} \hat{\Theta}_i) \end{aligned}$$

(where $\alpha(\Theta_i^{-1} \hat{\Theta}_i)$ denotes the unsigned angle of the rotation $\Theta_i^{-1} \hat{\Theta}_i$)

2.2 Iterative keyframe placement

We propose a greedy algorithm to iteratively add keyframes (denoted as a pair containing frame index i , controller vector c_i^* , and its derivative $c_i^{*'} \rightarrow$ to a timeline, given a motion sequence $(\Theta^i)_{0 \leq i \leq n}$.

We start with the timeline : $T = [(0, c_0^*, c_0^{*'}), (n, c_n^*, c_n^{*'})]$, such that $f(c_0^*) = \Theta^0$ and $f(c_n^*) = \Theta^n$.

Each frame $i \in \llbracket 1, n-1 \rrbracket$ is then surrounded by its closest keyframes $(l_i, c_{l_i}^*, c_{l_i}^{*'})$ and $(r_i, c_{r_i}^*, c_{r_i}^{*'})$. For each such frame, we compute the controller vector which minimizes the reconstruction error over the whole segment (interpolating using the neural rig):

Table 1: Performance of our neural rig on unseen test set

Metric	$\mathcal{L}_{\text{con}}$ (m)	$\mathcal{L}_{\text{rot}}$ (m)	$\mathcal{L}_{\text{pos}}$ (m)	$\mathcal{L}_{\text{ang}}$ (deg)
Value	$4.25 \cdot 10^{-2}$	$5.47 \cdot 10^{-2}$	$1.25 \cdot 10^{-2}$	4.73

Table 2: Results of our reconstruction on 150 unseen motions.

Method	MPJPE @10% (m)	Inference time (s)
Roberts	$7.56 \cdot 10^{-2}$	58.8
Ours	$5.27 \cdot 10^{-2}$	12.1

$$c_i^*, c_i^{*'} = \underset{c, c'}{\operatorname{argmin}} \sum_{j=l_i}^{r_i} \text{MPJPE} \left(g_\beta \left(\mathcal{I}_{l_i, c_{l_i}^*, c_{l_i}^{*'}, i, c, c', r_i, c_{r_i}^*, c_{r_i}^{*'}}(j) \right), J(\Theta^j) \right)$$

where MPJPE denotes the mean per-joint positional error of the skeletal joints, and $\mathcal{I}$ the Hermite cubic spline interpolation.

We then add to the timeline the frame k that improves reconstruction error the most: $T \leftarrow T + [(k, c_k^*, c_k^{*'})]$, until reaching a number of keyframes M , only recomputing c_i^* when necessary.

3 Results

We train our neural rig on the BEDLAM dataset [Black et al. 2023], and report its performance in Tables 1 and 2. We report the MPJPE with 10% of the keyframes, and inference time.

4 Conclusion

Our method effectively transforms dense MoCap animations into sparse timelines with keyframes on rig controllers. It achieves low reconstruction errors and generates animator-like data, enabling new machine learning applications where data is lacking. The neural rig serves as a proxy for an artist’s rig, facilitating editing by experts. Obtaining a differentiable formulation for any artist-created control rig would allow animators to retain their specific rigs.

References

- Michael J. Black, Priyanka Patel, Joachim Tesch, and Jinlong Yang. 2023. BEDLAM: A Synthetic Dataset of Bodies Exhibiting Detailed Lifelike Animated Motion. In *CVPR*. 8726–8737.
- Eyuphan Bulut and Tolga Capin. 2007. Key frame extraction from motion capture data by curve saliency. In *CASA*, Vol. 20.
- Xiaojing Chang, Pengfei Yi, and Qiang Zhang. 2016. Key frames extraction from human motion capture data based on hybrid particle swarm optimization algorithm. *Recent Developments in Intelligent Information and Database Systems* (2016), 335–342.
- Ke-Sen Huang, Chun-Fa Chang, Yu-Yao Hsu, and Shi-Nine Yang. 2005. Key probe: a technique for animation keyframe extraction. *The Visual Computer* 21 (2005).
- Ollie Johnston and Frank Thomas. 1995. *The Illusion of Life: Disney Animation*.
- Xian-mei Liu, Ai-min Hao, and Dan Zhao. 2013. Optimization-based key frame extraction for motion capture animation. *The visual computer* 29 (2013), 85–95.
- Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. 2015. SMPL: A Skinned Multi-Person Linear Model. *SIGGRAPH Asia* 34, 6 (Oct. 2015), 248:1–248:16.
- Clinton Mo, Kun Hu, Shaohui Mei, Zebin Chen, and Zhiyong Wang. 2021. Keyframe extraction from motion capture sequences with graph based deep reinforcement learning. In *ACM International Conference on Multimedia*. 5194–5202.
- Urs Ramer. 1972. An iterative procedure for the polygonal approximation of plane curves. *Computer graphics and image processing* 1, 3 (1972), 244–256.
- Richard Roberts, John P Lewis, Ken Anjyo, Jaewoo Seo, and Yeongho Seol. 2018. Optimal and interactive keyframe selection for motion capture. In *SIGGRAPH Asia 2018 Technical Briefs*. 1–4.
- Qiang Zhang, Xiang Xue, Dongsheng Zhou, and Xiaopeng Wei. 2014. Motion Keyframes extraction based on amplitude of distance characteristic curve. *IJCSIS* 7, 3 (2014), 506–514.
- Qiang Zhang, Shao-Pei Yu, Dong-Sheng Zhou, and Xiao-Peng Wei. 2013. An efficient method of key-frame extraction based on a cluster algorithm. *Journal of human kinetics* 39, 1 (2013), 5–14.